

Implement Client Script & UI Policy (Incident)

Phase 3 – Project Design

3.1 Purpose

The purpose of this phase is to design the ServiceNow components, configuration sequence, interaction between the UI Policy and Client Scripts, and the verification process before implementation.

The design provides a clear structure for how the **Implement Client Script & UI Policy (Incident)** project operates within the ServiceNow Incident Management application.

3.2 High-Level Architecture

The project follows the following logical process:

Incident Form → Impact Selection → UI Policy / Client Script → Field Control & Validation → Incident Update

For State control:

Incident List → State List Editing → onCellEdit Client Script → Validation → Change Blocked

Form-based State updates follow:

Incident Form → State Update → Form Submission → Incident Updated

3.3 System Design

The main ServiceNow components used in the project are:

Design Element	Configured Value
Platform	ServiceNow
Application	Incident Management
Main Table	Incident
UI Policy	High Impact Control
UI Policy Condition	Impact is 1 – High
UI Policy Action	Urgency

Design Element	Configured Value
onChange Client Script	Auto set urgency for high impact
onSubmit Client Script	Prevent save if Assigned To missing
onCellEdit Client Script	Prevent state change via list edit

3.4 UI Policy Design

The **High Impact Control** UI Policy is designed to activate when the Incident **Impact** field is set to **1 – High**.

Configuration

- **Name:** High Impact Control
- **Table:** Incident
- **Condition:** Impact is 1 – High
- **Active:** Yes
- **Reverse if false:** Enabled

When the condition is true, the **Assignment Group** field becomes mandatory.

When the condition becomes false, the configured behavior is reversed.

3.5 UI Policy Action Design

The UI Policy Action is associated with the **High Impact Control** UI Policy.

Configuration

- **Field:** Urgency
- **Mandatory:** Leave alone
- **Visible:** Leave alone
- **Read-only:** Yes

When Impact is High, the Urgency field becomes read-only.

3.6 Client Script Design

The project uses three Client Scripts to provide different types of Incident behavior.

3.6.1 onChange Client Script

Name: Auto set urgency for high impact

Table: Incident

Type: onChange

Field: Impact

The script is designed to run when the Impact field changes.

When Impact is set to High:

- Urgency is automatically set to High.
- An informational message is displayed.

3.6.2 onSubmit Client Script

Name: Prevent save if Assigned To missing

Table: Incident

Type: onSubmit

The script checks:

Impact = High + Assigned To is empty

When both conditions are satisfied, the Incident submission is prevented and an error message is displayed.

When a valid Assigned To value is provided, the Incident can be submitted.

3.6.3 onCellEdit Client Script

Name: Prevent state change via list edit

Table: Incident

Type: onCellEdit

Field: State

The script is designed to run when the State field is edited directly from the Incident list.

The list-based State change is prevented and a warning message is displayed.

Users can instead open the Incident record and modify State through the Incident form.

3.7 Flow Design

The overall configuration can be represented as follows:

Impact = High



High Impact Control UI Policy



Assignment Group becomes Mandatory



Urgency becomes Read-only



onChange Client Script sets Urgency to High



User provides Assigned To



onSubmit Client Script validates submission



Incident is Saved

For State control:

Incident List



User attempts State change



onCellEdit Client Script



Warning displayed



State list edit blocked

3.8 Verification Design

The designed configuration will be verified using the following conditions:

1. Set Impact to **1 – High**.
2. Verify that Assignment Group becomes mandatory.
3. Verify that Urgency becomes read-only.
4. Verify that Urgency is automatically set to High.
5. Attempt submission without Assigned To.
6. Verify that submission is prevented.
7. Provide a valid Assigned To value.
8. Verify that the Incident can be saved.
9. Attempt to change State directly from the Incident list.
10. Verify that the list-based State change is blocked.
11. Change State through the Incident form.
12. Verify that the form-based update is successful.

3.9 Design Output

The project design defines the complete logical structure of the **Implement Client Script & UI Policy (Incident)** solution.

The design establishes how the UI Policy, UI Policy Action, onChange Client Script, onSubmit Client Script, and onCellEdit Client Script work together to control Incident behavior.

This design provides the foundation for the next phase, **Phase 4 – Project Planning**, where implementation activities, responsibilities, sequence, and completion checkpoints are organized.